9 Configuring CSRF protection

This chapter covers

- Understanding CSRF attacks
- Implementing CSRF protection
- Customizing CSRF protection

You have learned about the filter chain and its purpose in the Spring Security architecture. We worked on several examples in chapter 5, where we customized the filter chain. But Spring Security also adds its own filters to the chain. This chapter discusses the filter that configures CSRF (cross-site request forgery) protection. You'll learn to customize the filters to make a perfect fit for your scenarios.

You have probably observed that in most of the examples up to now, we only implemented our endpoints with HTTP GET. Moreover, when we needed to configure HTTP POST, we also had to add a supplementary instruction to the configuration to disable CSRF protection. The reason why you can't call an endpoint with HTTP POST directly is because of CSRF protection, which is enabled by default in Spring Security.

We'll now discuss CSRF protection and when to use it in your applications. CSRF is a widespread type of attack, and vulnerable applications can force users to execute unwanted actions on a web application following authentication. You don't want the applications you develop to be CSRF vulnerable and allow attackers to trick your users into executing unwanted actions.

Because it's essential to understand how to mitigate these vulnerabilities, we start by reviewing what CSRF is and how it works. We then discuss the CSRF token mechanism that Spring Security uses to mitigate CSRF vulnerabilities. We continue with obtaining a token and using it to call an endpoint with the HTTP POST method. We prove this using a small application with REST endpoints. Once you learn how Spring Security implements its CSRF token mechanism, we discuss how to use it in real-world application scenarios. Finally, you'll learn possible customizations of the CSRF token mechanism in Spring Security.

9.1 How CSRF protection works in Spring Security

This section discusses how Spring Security implements CSRF protection. It is essential to understand first the underlying mechanism of CSRF protection. I encounter many situations in which misunderstanding the way CSRF protection works leads to its misuse, either because it is disabled in scenarios where it should be enabled or the other way around. Like any other feature in a framework, you must use it correctly to bring value to your applications.

For example, consider this scenario (figure 9.1): you are at work, using a web tool to store and manage your files. With this tool, in a web interface, you can add new files, add new versions for your records, and even delete them. You receive an email asking you to open a page for a specific reason (for example, a promotion at your favorite store). You open the page, but the page is blank, or it redirects you to a known website (the online shop of your favorite store). You go back to your work finding that all your files are gone!

What happened? You were logged into your work application so you could manage your files. When you add, change, or delete a file, the web page you interact with calls some endpoints from the server to execute these operations. When you opened the foreign page by clicking the unknown link in the email, that page called your app's backend and executed actions on your behalf (i.e., it deleted your files).

It could do that because you logged in previously, so the server trusted that the actions came from you. You might think that someone couldn't trick you so easily into clicking a link from a foreign email or message, but trust me, this happens to many people. Most web app users aren't aware of security risks. So it's wiser if you, who know all the tricks, protect your users and build secure apps rather than rely on your apps' users to protect themselves.

CSRF attacks assume that a user is logged into a web application. The attacker tricks users into opening a page containing scripts that execute actions in the same application the user was working on. Because the user has already logged in (as we've assumed from the beginning), the forgery code can now impersonate the user and do actions on their behalf.

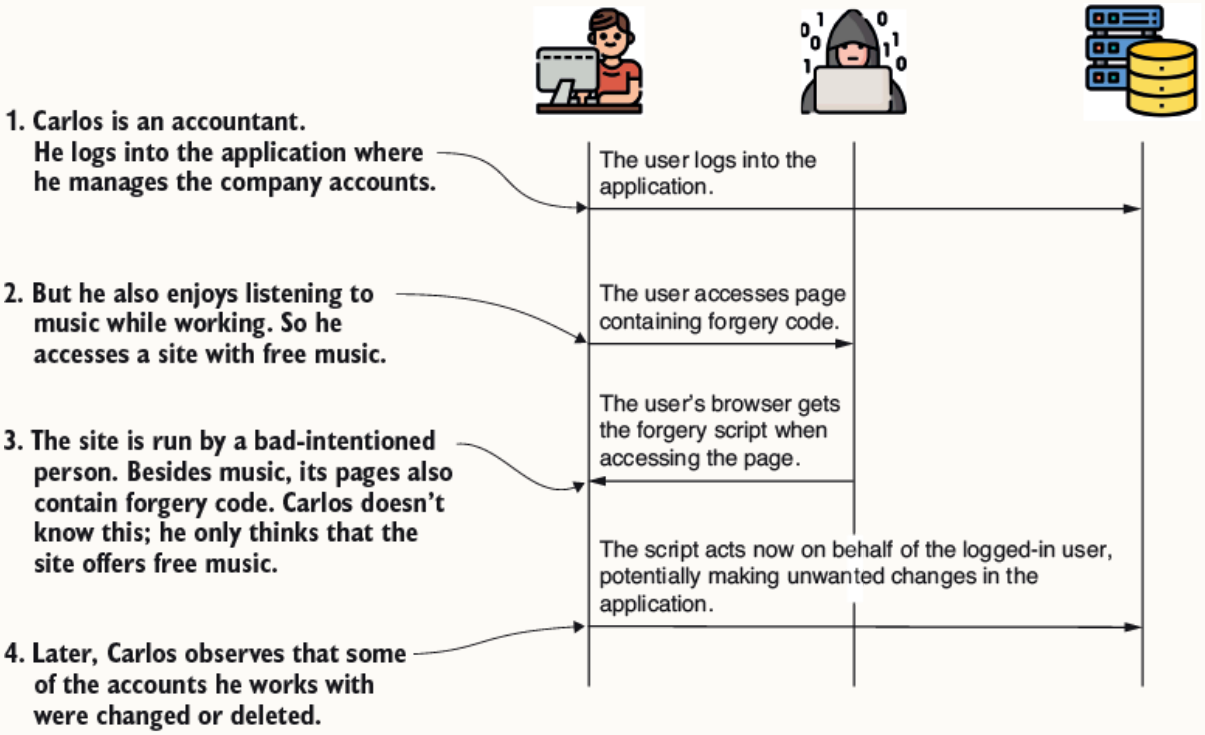


Figure 9.1 After the user logs into their account, they access a page containing forgery code. This code impersonates the user and can execute unwanted actions on behalf of the user.

How do we protect our users from such scenarios? CSRF protection aims to ensure that only the frontend of web applications can perform mutating operations (by convention, HTTP methods other than GET, HEAD, TRACE, or OPTIONS). Then a foreign page, like the one in our example, can't act on behalf of the user.

How can we achieve this? What you know for sure is that before being able to do any action that could change data, a user must send a request using HTTP GET to see the web page at least once. When this happens, the application generates a unique token. The application now accepts only requests for mutating operations (POST, PUT, DELETE, etc.) that contain this unique value in the header.

The application considers that knowing the token's value is proof that it is the app itself making the mutating request and not another system. Any page containing mutating calls, such as POST, PUT, DELETE, and so on, should receive through the CSRF token the response, and the page must use this token when making mutating calls.

The starting point of CSRF protection is a filter in the filter chain called `CsrfFilter`. The `CsrfFilter` intercepts requests and allows all those that use these HTTP methods: GET, HEAD, TRACE, and OPTIONS. For all other requests, the filter expects to receive a header containing a token. If this header does not exist or contains an incorrect token value, the application rejects the request and sets the response status to HTTP 403 Forbidden.

What is this token, and where does it come from? These tokens are nothing more than string values. You must add the token in the request's header when you use any method other than GET, HEAD, TRACE, or OPTIONS. If you don't do this, the application doesn't accept the request, as presented in figure 9.2.

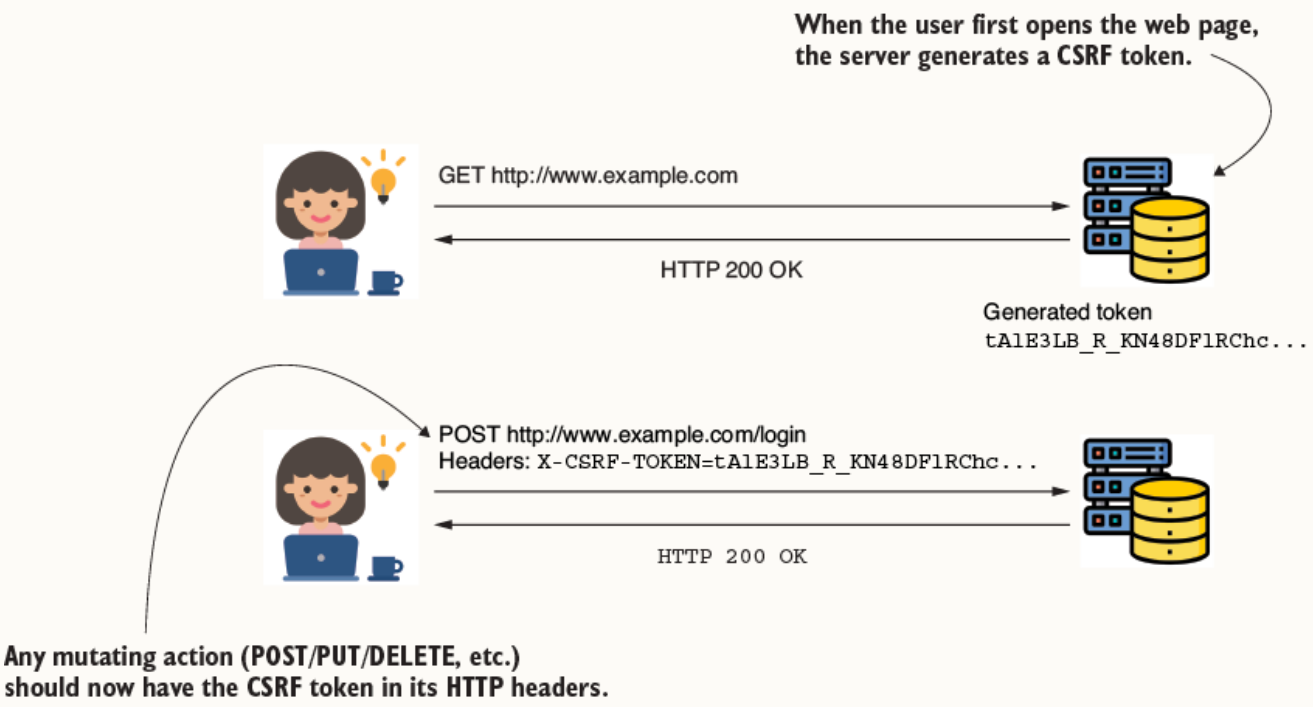


Figure 9.2 To make a POST request, the client needs to add a header containing the CSRF token. The application generates a CSRF token when the page is loaded (via a GET request), and the token is added to all requests that can be made from the loaded page. This way, only the loaded page can make mutating requests.

The `CsrfFilter` (figure 9.3) uses a component named `CsrfTokenRepository` to manage the CSRF token values that generate new tokens, store tokens, and eventually invalidate these. By default, the `CsrfTokenRepository` stores the token on the HTTP session and generates the tokens as random string values. In most cases, this is enough, but as you'll learn in section 9.3, you can use your own implementation of `CsrfTokenRepository` if the default one doesn't apply to the requirements you need to implement.

In this section, I explained how CSRF protection works in Spring Security with plenty of text and figures. But I want to enforce your understanding with a small code example as well. You'll find this code as part of the project named ssia-ch9-ex1. Let's create an application that exposes two endpoints. We can call one of these with HTTP GET and the other with HTTP POST.

As you know by now, you are not able to call endpoints with POST directly without disabling CSRF protection. In this example, you learn how to call the POST endpoint without disabling CSRF protection. You need to obtain the CSRF token so that you can use it in the header of the call, which you do with HTTP POST.

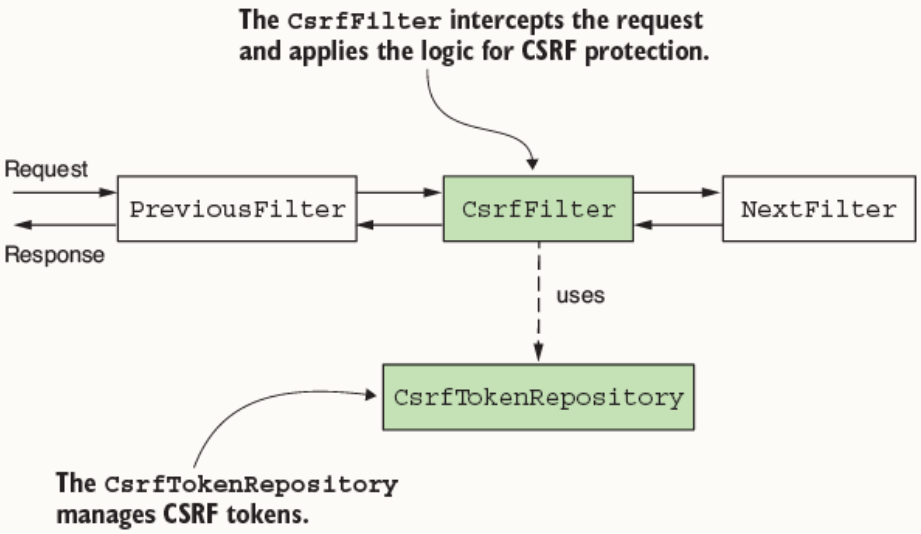


Figure 9.3 The `CsrfFilter` is one of the filters in the filter chain. It receives the request and eventually forwards it to the next filter in the chain. To manage CSRF tokens, `CsrfFilter` uses a `CsrfTokenRepository`.

As you learn from this example, the `CsrfFilter` adds the generated CSRF token to the attribute of the HTTP request named `_csrf` (figure 9.4). If we know this, we know that after the `CsrfFilter`, we can find this attribute and take the token value from it. For this small application, we choose to add a custom filter after the `CsrfFilter`, as you learned in chapter 5. You use this custom filter to print the CSRF token in the console of the application that the app generates when we call the endpoint using HTTP GET. We can then copy the token value from the console and use it to make the mutating call with HTTP POST. In listing 9.1, you can find the definition of the controller class with the two endpoints that we use for a test.

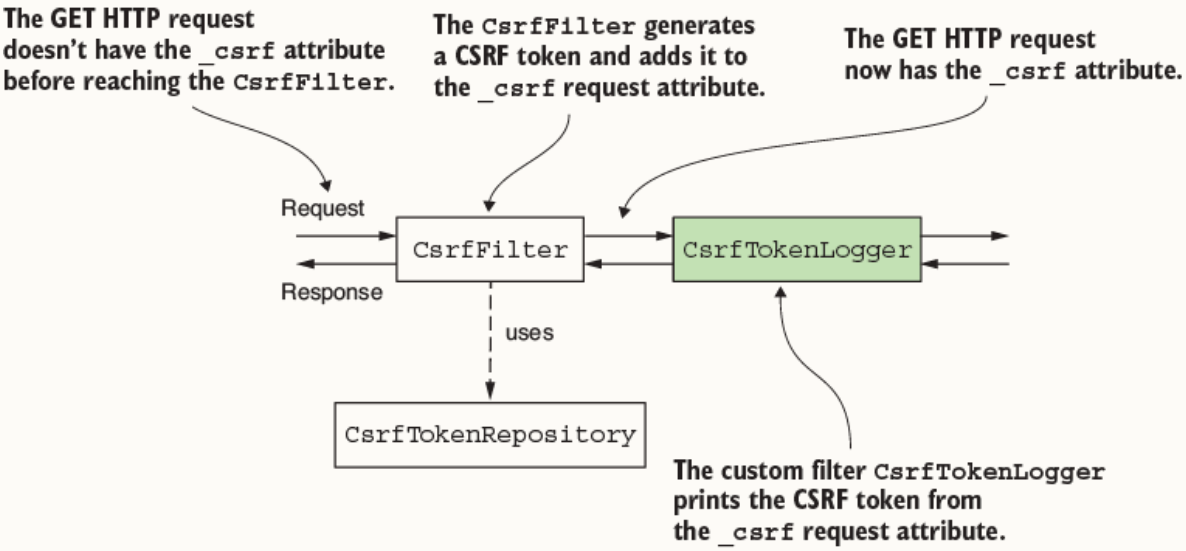


Figure 9.4 Inserting the `CsrfTokenLogger` (highlighted) following the `CsrfFilter` allows it to retrieve the token's value from the request's `_csrf` attribute, which is where the `CsrfFilter` deposits it. The `CsrfTokenLogger` outputs the CSRF token onto the application's console, from where it can be retrieved for use in making an HTTP POST request to an endpoint.

Listing 9.1 The controller class with two endpoints

```
@RestController
public class HelloController {

  @GetMapping("/hello")
  public String getHello() {
    return "Get Hello!";
  }

  @PostMapping("/hello")
  public String postHello() {
    return "Post Hello!";
  }
}
```

Listing 9.2 defines the custom filter we use to print the value of the CSRF token in the console. I named the custom filter `CsrfTokenLogger`. When called, the filter obtains the value of the CSRF token from the `_csrf` request attribute and prints it in the console. The name of the request attribute, `_csrf`, is where the `CsrfFilter` sets the value of the generated CSRF token as an instance of the class `CsrfToken`. This instance of `CsrfToken` contains the string value of the CSRF token. You can obtain it by calling the `getToken()` method.

Listing 9.2 The definition of the custom filter class

```
public class CsrfTokenLogger implements Filter {

  private Logger logger =
      Logger.getLogger(CsrfTokenLogger.class.getName());
```

```
  @Override
  public void doFilter(
    ServletRequest request,
    ServletResponse response,
    FilterChain filterChain)
      throws IOException, ServletException {

      CsrfToken o =
       (CsrfToken) request.getAttribute("_csrf");     ①

      logger.info("CSRF token " + token.getToken());

      filterChain.doFilter(request, response);
  }
}
```

① Takes the value of the token from the _csrf request attribute and prints it in the console

In the configuration class, we add the custom filter. The next listing presents the configuration class. Observe that I don't disable CSRF protection in the listing.

Listing 9.3 Adding the custom filter in the configuration class

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain configure(HttpSecurity http)
    throws Exception {

    http.addFilterAfter(
          new CsrfTokenLogger(), CsrfFilter.class)
        .authorizeHttpRequests(
          c -> c.anyRequest().permitAll()
        );

    return http.build();
  }
}
```

We can now test the endpoints. We begin by calling the endpoint with HTTP GET. Because the default implementation of the `CsrfTokenRepository` interface uses the HTTP session to store the token value on the server side, we also need to remember the session ID. For this reason, I add the `-v` flag to the call so that I can see more details from the response, including the session ID. Calling the endpoint

```
curl -v http://localhost:8080/hello
```

returns this (truncated) response:

```
...
< Set-Cookie: JSESSIONID=21ADA55E10D70BA81C338FFBB06B0206;
...
Get Hello!
```

Following the request in the application console, you can find a log line that contains the CSRF token:

```
INFO 21412 --- [nio-8080-exec-1] c.l.ssia.filters.CsrfTokenLogger : CSRF token tAlE3LB_R_KN48DFlRChc...
```

NOTE You might ask yourself how clients get the CSRF token. They can neither guess it nor read it in the server logs. I designed this example so that it's easier for you to understand how CSRF protection implementation works. As you'll find in section 9.2, the backend application has the responsibility to add the value of the CSRF token in the HTTP response to be used by the client.

If you call the endpoint using the HTTP POST method without providing the CSRF token, the response status is 403 Forbidden, as this command line shows:

```
curl -XPOST http://localhost:8080/hello
```

The response body is

```
{
   "status":403,
   "error":"Forbidden",
   "message":"Forbidden",
   "path":"/hello"
}
```

But if you provide the correct value for the CSRF token, the call is successful. You also need to specify the session ID (`JSESSIONID`) because the default implementation of `CsrfTokenRepository` stores the value of the CSRF token on the session:

```
curl -X POST   http://localhost:8080/hello
-H 'Cookie: JSESSIONID=21ADA55E10D70BA81C338FFBB06B0206'
-H 'X-CSRF-TOKEN: tAlE3LB_R_KN48DFlRChc...'
```

The response body is

```
Post Hello!
```

9.2 Using CSRF protection in practical scenarios

In this section, we discuss applying CSRF protection in practical situations. Now that you know how CSRF protection works in Spring Security, you need to know where you should use it in the real world. Which kinds of applications need to use CSRF protection?

You use CSRF protection for web apps running in a browser, where you should expect that mutating operations can be done by the browser that loads the displayed content of the app. The most basic example I can provide here is a simple web application developed on the standard Spring MVC flow. We already made such an application when discussing form login in chapter 6, and that web app actually used CSRF protection. Did you notice that the login operation in that application used HTTP POST? Then why didn't we need to do anything explicitly about CSRF in that case? The reason why we didn't observe this was because we didn't develop any mutating operation there.

For the default form login, Spring Security correctly applies CSRF protection for us. The framework takes care of adding the CSRF token to the login request. Let's now develop a similar application to look more closely at how CSRF protection works. As figure 9.5 shows, in this section we

- Build an example of a web application with the login form
- Look at how the default implementation of the login uses CSRF tokens
- Implement an HTTP POST call from the main page

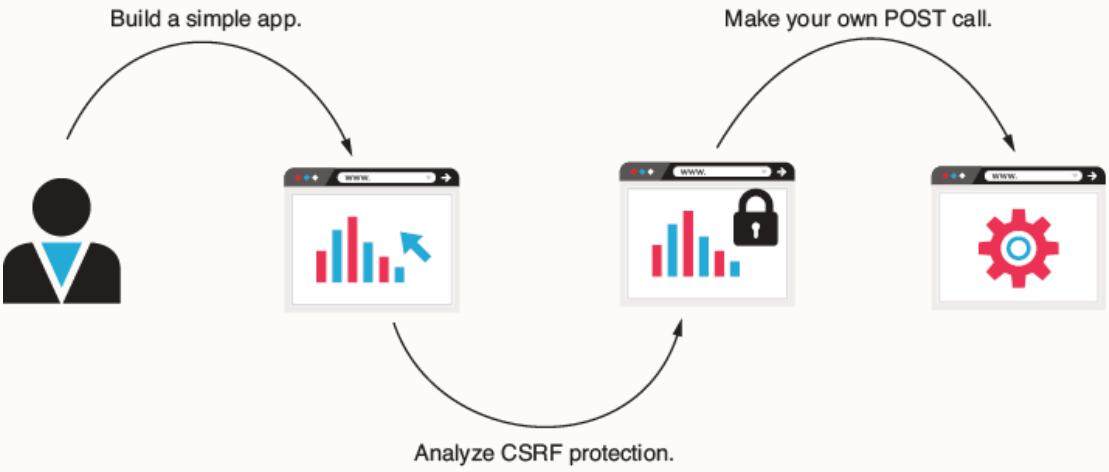


Figure 9.5 The plan. In this section, we start by building and analyzing a simple app to understand how Spring Security applies CSRF protection, and then we write our own POST call.

In this example application, you'll notice that the HTTP POST call won't work until we correctly use the CSRF tokens, and here, you'll learn how to apply the CSRF tokens in a form on such a web page. To implement this application, we start by creating a new Spring Boot project. You can find this example in the project ssia-ch9-ex2. The next code snippet presents the needed dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Next, of course, we need to configure the form login and at least one user. The following listing presents the configuration class, which defines the `UserDetailsService`, adds a user, and configures the `formLogin` method.

Listing 9.4 The definition of the configuration class

```
public class ProjectConfig {

 @Bean                                ①
 public UserDetailsService uds() {
  var uds = new InMemoryUserDetailsManager();

  var u1 = User.withUsername("mary")
           .password("12345")
```

```
            .authorities("READ")
            .build();

    uds.createUser(u1);

    return uds;
  }

  @Bean                                ②
  public PasswordEncoder passwordEncoder() {
    return NoOpPasswordEncoder.getInstance();
  }

  @Bean                                ③
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.formLogin(
      c -> c.defaultSuccessUrl("/main", true)
    );


    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated()
    );

    return http.build();
  }
}
```

① Adds a UserDetailsService bean managing one user to test the application

② Adds a PasswordEncoder

③ Creates a bean of type SecurityFilterChain to set the form login authentication method and specifies that only authenticated users can access any of the endpoints

We add a controller class for the main page in a package named controllers and in a main.html file in the resources/templates folder of the Maven project. The main.html file can remain empty for the moment because on the first execution of the application, we only focus on how the login page uses the CSRF tokens. The following listing presents the `MainController` class, which serves the main page.

Listing 9.5 The definition of the `MainController` class

```
@Controller
public class MainController {

  @GetMapping("/main")
  public String main() {
    return "main.html";
  }
}
```

After running the application, you can access the default login page. If you inspect the form using the inspect element function of your browser, you can observe that the default implementation of the login form sends the CSRF token.

This is why your login works with CSRF protection enabled even if it uses an HTTP POST request! Figure 9.6 shows how the login form sends the CSRF token through hidden input.

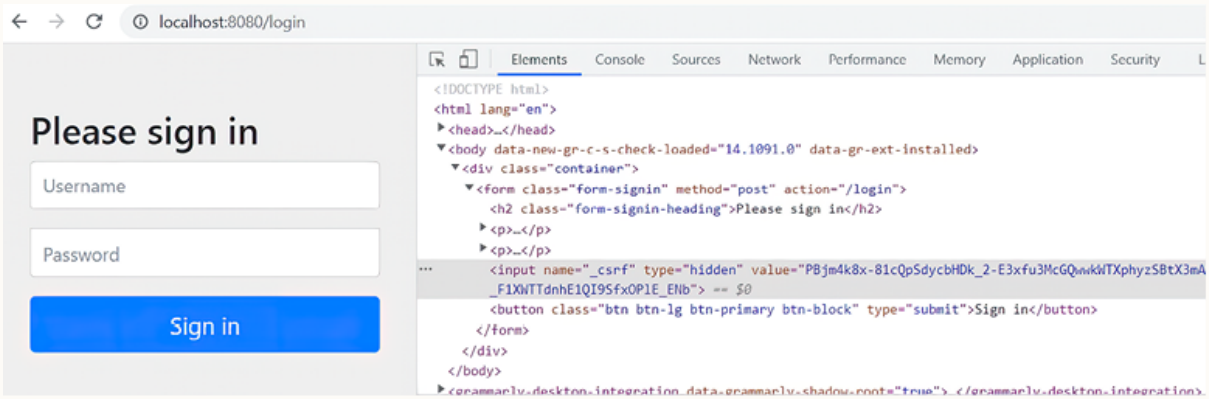


Figure 9.6 The default form login uses a hidden input to send the CSRF token in the request. This is why the login request that uses an HTTP POST method works with CSRF protection enabled.

But what about developing our own endpoints that use POST, PUT, or DELETE as HTTP methods? For these, we must take care of sending the value of the CSRF token if CSRF protectionW is enabled. To test this, let's add an endpoint using HTTP POST to our application. We call this endpoint from the main page, and we create a second controller for this, called `ProductController`. Within this controller, we define an endpoint, /product/add, that uses HTTP POST. Next, we use a form on the main page to call this endpoint. The following listing defines the `ProductController` class.

Listing 9.6 The definition of the `ProductController` class

```
@Controller
@RequestMapping("/product")
public class ProductController {

  private Logger logger =
    Logger.getLogger(ProductController.class.getName());

  @PostMapping("/add")
  public String add(@RequestParam String name) {
    logger.info("Adding product " + name);
    return "main.html";
  }
}
```

The endpoint receives a request parameter and prints it in the application console. The following listing shows the definition of the form defined in the main.html file.

Listing 9.7 The definition of the form in the main.html page

```
<form action="/product/add" method="post">
  <span>Name:</span>
  <span><input type="text" name="name" /></span>
  <span><button type="submit">Add</button></span>
</form>
```

Now you can rerun the application and test the form. You'll observe that when submitting the request, a default error page is displayed, which confirms an HTTP 403 Forbidden status on the response from the server (figure 9.7). The reason for this status is the absence of the CSRF token.



Figure 9.7 If the CSRF token is not included, the server will reject any requests made using the HTTP POST method. The user will be rerouted to a standard error page that displays a status of HTTP 403 Forbidden in the response.

To solve this problem and make the server allow the request, we need to add the CSRF token to the request done through the form. An easy way to do this is to use a hidden input component, as you saw in the default form login. This can be implemented as presented in the following listing.

Listing 9.8 Adding the CSRF token to the request done through the form

```
<form action="/product/add" method="post">
  <span>Name:</span>
  <span><input type="text" name="name" /></span>
  <span><button type="submit">Add</button></span>

  <input type="hidden"                    ①
      th:name="${_csrf.parameterName}"    ②
      th:value="${_csrf.token}" />        ②
</form>
```

① Uses hidden input to add the request to the CSRF token

② The "th" prefix enables Thymeleaf to print the token value.

NOTE In the example, we use Thymeleaf because it provides a straightforward way to obtain the request attribute value in the view. In our case, we need to print the CSRF token. Remember that the `CsrfFilter` adds the value of the token in the `_csrf` attribute of the request. It's not mandatory to do this with Thymeleaf. You can use any alternative of your choice to print the token value to the response.

After rerunning the application, you can test the form again. This time, the server accepts the request, and the application prints the log line in the console, proving that the execution is successful. Furthermore, if you inspect the form, you can find the hidden input with the value of the CSRF token (figure 9.8).

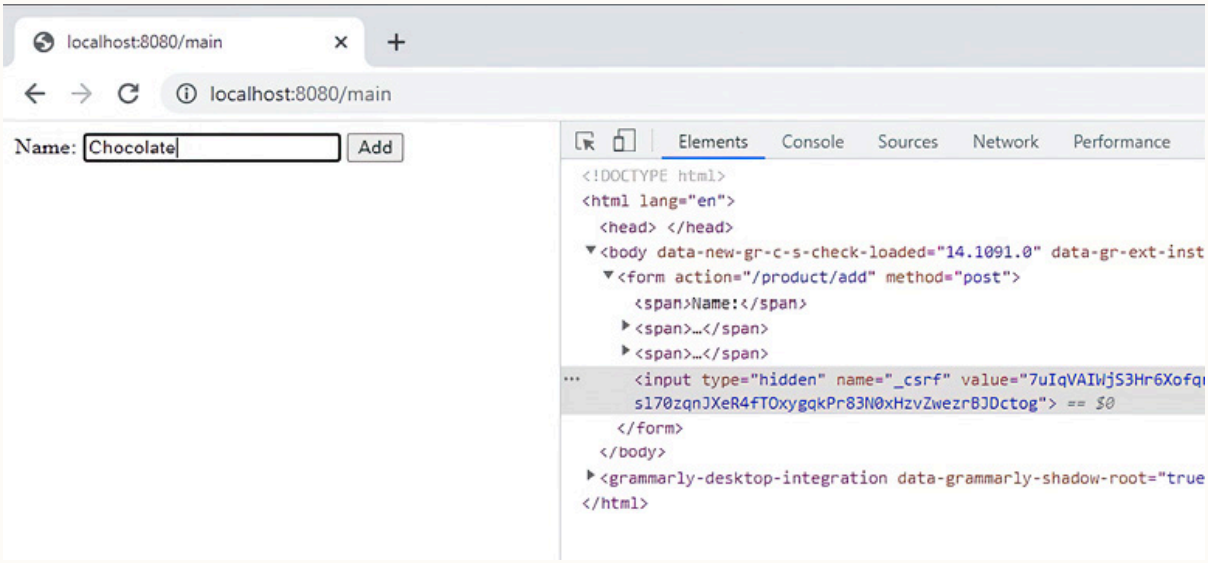


Figure 9.8 The form defined on the main page now sends the value for the CSRF token in the request. This way, the server allows the request and executes the controller action. In the source code for the page, you can now find the hidden input used by the form to send the CSRF token in the request.

After submitting the form, you should find in the application console a line similar to this one:

```
INFO 20892 --- [nio-8080-exec-7] c.l.s.controllers.ProductController    : Adding product Chocolate
```

Of course, for any action or asynchronous JavaScript request your page uses to call a mutating action, you need to send a valid CSRF token. This is the most common way used by an application to make sure the request doesn't come from a third party. A third-party request could try to impersonate the user to execute actions on their behalf.

CSRF tokens work well in an architecture where the same server is responsible for both the frontend and the backend, mainly for its simplicity. But CSRF tokens don't work well when the client is independent of the backend solution it consumes. This scenario happens when you have a mobile application as a client or a web frontend developed independently. A web client developed with a framework such as Angular, ReactJS, or Vue.js is ubiquitous in web application architectures, and this is why you need to know how to implement the security approach for these cases as well. We'll discuss these kinds of designs in part 4 of this book.

In chapters 13 through 16, you'll learn to implement the OAuth 2 specification, which has excellent advantages in decoupling the component. This makes the authentication from the resources for which the application authorizes the client.

NOTE It might look like a trivial mistake, but in my experience, I see it too many times in applications—never use HTTP GET with mutating operations! Do not implement behavior that changes data and allows it to be called with an HTTP GET endpoint. Remember that calls to HTTP GET endpoints don't require a CSRF token.

9.3 Customizing CSRF protection

In this section, you learn how to customize the CSRF protection solution offered by Spring Security. Because applications have various requirements, any implementation provided by a framework needs to be flexible enough to be easily adapted to different scenarios. The CSRF protection mechanism in Spring Security is no exception. In this section, the examples let you apply the most frequently encountered needs in the customization of the CSRF protection mechanism. These are

- Configuring paths for which CSRF applies
- Managing CSRF tokens

We use CSRF protection only when the page that consumes resources produced by the server is itself generated by the same server. It can be a web application where the consumed endpoints are exposed by a different origin, as we discussed in section 9.2, or a mobile application. In the case of mobile applications, you can use the OAuth 2 flow, which we'll discuss in chapters 13 through 16.

By default, CSRF protection applies to any path for endpoints called with HTTP methods other than GET, HEAD, TRACE, or OPTIONS. You already know from chapter 5 how to completely disable CSRF protection. But what if you want to disable it only for some of your application paths? You can do this configuration quickly with a `Customizer` object, similar to the way we customized HTTP Basic for form-login methods in chapter 6.

Here, we create a new project and add only the web and security dependencies, as presented in the next code snippet. You can find this example in the project ssia-ch9-ex3. Here are the dependencies:

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

In this application, we add two endpoints called with HTTP POST, but we want to exclude one of these from using CSRF protection (figure 9.9). Listing 9.9 defines the controller class for this, which I name `HelloController`.

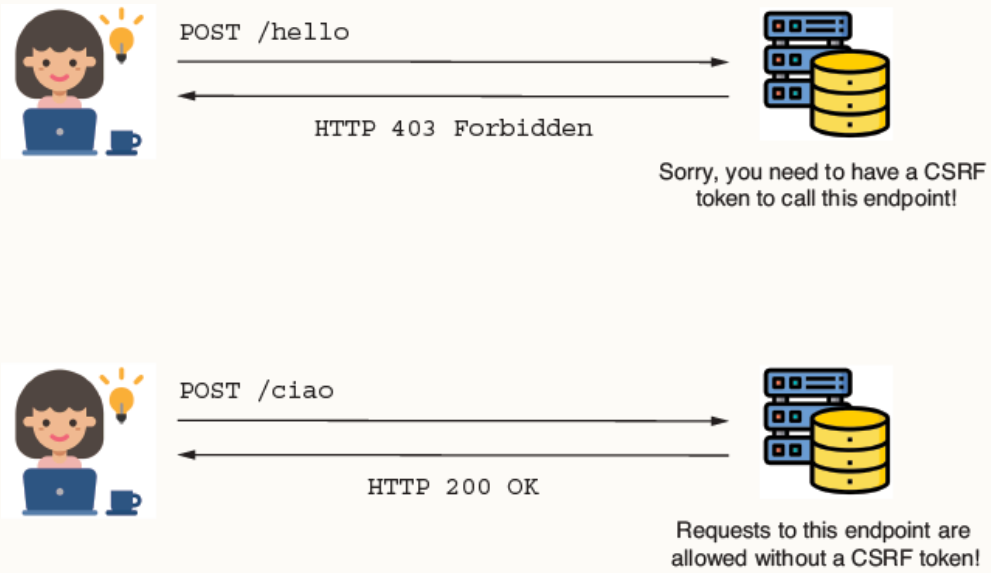


Figure 9.9 The application requires a CSRF token for the /hello endpoint called with HTTP POST but allows HTTP POST requests to the /ciao endpoint without a CSRF token.

Listing 9.9 The definition of the `HelloController` class

```
@RestController
public class HelloController {

 @PostMapping("/hello")        ①
 public String postHello() {
  return "Post Hello!";
 }

 @PostMapping("/ciao")         ②
```

```
  public String postCiao() {
    return "Post Ciao";
  }
}
```

① The /hello path remains under CSRF protection. You can't call the endpoint without a valid CSRF token.

② The /ciao path can be called without a CSRF token.

To make customizations on CSRF protection, you can use the `csrf()` method of the `HttpSecurity` object in the `securityFilterChain()` method with a `Customizer` object. The next listing presents this approach.

Listing 9.10 A `Customizer` object for the configuration of CSRF protection

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.csrf(c -> {                    ①
      c.ignoringRequestMatchers("/ciao");
    });

    http.authorizeHttpRequests(
        c -> c.anyRequest().permitAll()
    );

    return http.build();
  }
```

① The parameter of the lambda expression is a CsrfConfigurer. By calling its methods, you can configure CSRF protection in various ways.

Calling the `ignoringRequestMatchers(String paths)` method, you can specify the path expressions representing the paths that you want to exclude from the CSRF protection mechanism. A more general approach is to use a `RequestMatcher`. Using this allows you to apply the exclusion rules with regular path expressions as well as with regexes (regular expressions). When using the `ignoringRequestMatchers()` method of the `CsrfCustomizer` object, you can provide any `RequestMatcher` as a parameter. The next code snippet shows how to use the `ignoringRequestMatchers()` method with an `MvcRequestMatcher` instead of using `ignoringRequestMatchers()` with a path given as a `String` value:

```
HandlerMappingIntrospector i = new HandlerMappingIntrospector();
MvcRequestMatcher r = new MvcRequestMatcher(i, "/ciao");
c.ignoringRequestMatchers(r);
```

Or you can similarly use a regex matcher:

```
String pattern = ".*[0-9].*";
String httpMethod = HttpMethod.POST.name();
RegexRequestMatcher r = new RegexRequestMatcher(pattern, httpMethod);
c.ignoringRequestMatchers(r);
```

Another need often found in the requirements of the application is customizing the management of CSRF tokens. As you learned, by default, the application stores CSRF tokens in the HTTP session on the server side. This simple approach is suitable for small applications, but it's not great for applications that serve a large number of requests and that require horizontal scaling. The HTTP session is stateful and reduces the scalability of the application.

Let's suppose you want to change the way the application manages tokens and store them somewhere in a database rather than in the HTTP session. Spring Security offers three contracts that you need to implement to do this:

- `CsrfToken` —Describes the CSRF token itself
- `CsrfTokenRepository` —Describes the object that creates, stores, and loads CSRF tokens
- `CsrfTokenRequestHandler` —Describes the object that manages the way in which the generated CSRF token is set upon the HTTP request

The `CsrfToken` object has three main characteristics that you need to specify when implementing the contract (listing 9.11 defines the `CsrfToken` contract):

- The name of the header in the request that contains the value of the CSRF token (default named `X-CSRF-TOKEN`)
- The name of the attribute of the request that stores the value of the token (default named `_csrf`)
- The value of the token

Listing 9.11 The definition of the `CsrfToken` interface

```
public interface CsrfToken extends Serializable {

  String getHeaderName();
  String getParameterName();
  String getToken();
}
```

Generally, you only need the instance of the `CsrfToken` type to store the three details in the attributes of the instance. For this functionality, Spring Security offers an implementation called `DefaultCsrfToken` that we also use in our example. `DefaultCsrfToken` implements the `CsrfToken` contract and creates immutable instances containing the required values: the name of the request attribute and header, and the token itself.

The interface `CsrfTokenRepository` is the contract that represents the component managing CSRF tokens. To change the way the application manages the tokens, you need to implement the `CsrfTokenRepository` interface, which allows you to plug your custom implementation into the framework. Let's change the current application we use in this section to add a new implementation for `CsrfTokenRepository`, which stores the tokens in a database. Figure 9.10 presents the components we implement for this example and the link between them.

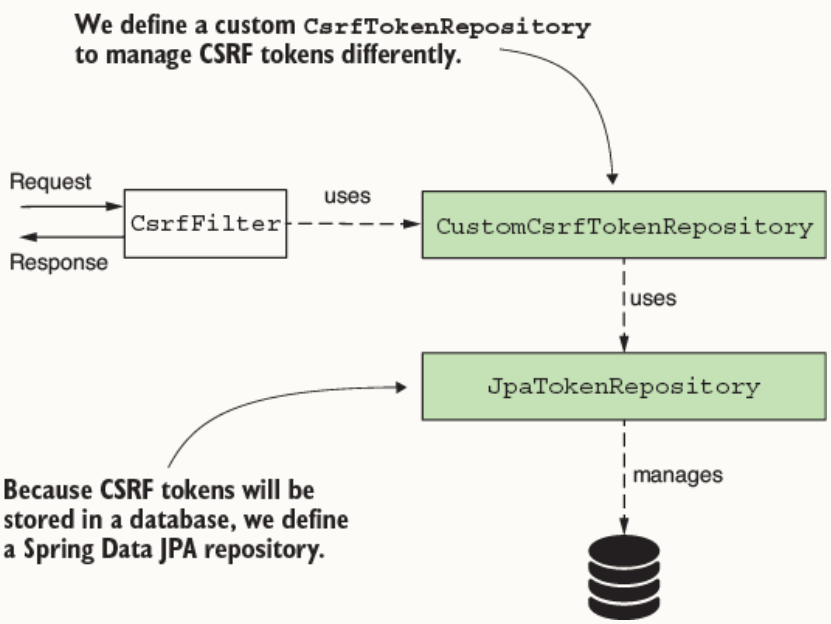


Figure 9.10 The `CsrfToken` uses a custom implementation of `CsrfTokenRepository`. This custom implementation uses a `JpaRepository` to manage CSRF tokens in a database.

In our example, we use a table in a database to store CSRF tokens. We assume the client has an ID to identify themselves uniquely. The application needs this identifier to obtain the CSRF token and validate it. Generally, this unique ID would be obtained during login and should be different each time the user logs in. This strategy of managing tokens is similar to storing them in memory. In this case, you use a session ID. Thus, the new identifier for this example merely replaces the session ID.

An alternative to this approach would be to use CSRF tokens with a defined lifetime. With such an approach, tokens expire after a time you define. You can store tokens in the database without linking them to a specific user ID. You only need to check if a token provided via an HTTP request exists and is not expired to decide whether you allow that request.

EXERCISE Once you finish with this example where we use an identifier to which we assign the CSRF token, implement the second approach where you use CSRF tokens that expire.

To make our example shorter, we only focus on the implementation of the `CsrfTokenRepository`, and we need to consider that the client already has a generated identifier. To work with the database, we need to add a couple more dependencies to the pom.xml file:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

In the application.properties file, we need to add the properties for the database connection:

```
spring.datasource.url=jdbc:mysql://localhost/spring
➥?useLegacyDatetimeCode=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.sql.init.mode=always
```

To allow the application to create the needed table in the database at the start, you can add the schema.xml file in the resources folder of the project. This file should contain the query for creating the table:

```
CREATE TABLE IF NOT EXISTS `spring`.`token` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `identifier` VARCHAR(45) NULL,
  `token` TEXT NULL,
PRIMARY KEY (`id`));
```

We use Spring Data with a JPA implementation to connect to the database, so we need to define the entity class and the `JpaRepository` class. In a package named entities, we define the JPA entity as presented in the following listing.

Listing 9.12 The definition of the JPA entity class

```
@Entity
public class Token {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private int id;

 private String identifier;    ①
 private String token;         ②

 // Omitted code

}
```

① The identifier of the client

② The CSRF token generated by the application for the client

The `JpaTokenRepository`, which is our `JpaRepository` contract, can be defined as shown in the following listing. The only method you need is `findTokenByIdentifier()`, which gets the CSRF token from the database for a specific client.

Listing 9.13 The definition of the `JpaTokenRepository` interface

```
public interface JpaTokenRepository
 extends JpaRepository<Token, Integer> {

 Optional<Token> findTokenByIdentifier(String identifier);
}
```

With access to the implemented database, we can now start writing the `CsrfTokenRepository` implementation, which I call `CustomCsrfTokenRepository`. The next listing defines this class, which overrides the three methods of `CsrfTokenRepository`.

Listing 9.14 The implementation of the `CsrfTokenRepository` contract

```
@Component
public class CustomCsrfTokenRepository implements CsrfTokenRepository {

 private final JpaTokenRepository jpaTokenRepository;
```

```
  // Omitted constructor

  @Override
  public CsrfToken generateToken(
    HttpServletRequest httpServletRequest) {
    // ...
  }

  @Override
  public void saveToken(
    CsrfToken csrfToken,
    HttpServletRequest httpServletRequest,
    HttpServletResponse httpServletResponse) {
    // ...
  }

  @Override
  public CsrfToken loadToken(
    HttpServletRequest httpServletRequest) {
    // ...
  }
}
```

`CustomCsrfTokenRepository` injects an instance of `JpaTokenRepository` from the Spring context to gain access to the database. `CustomCsrfTokenRepository` uses this instance to retrieve or to save the CSRF tokens in the database. The CSRF protection mechanism calls the `generateToken()` method when the application needs to generate a new token. Listing 9.15 illustrates implementation of this method for our exercise. We use the `UUID` class to generate a new random UUID value, and we keep the same names for the request header and attribute, `X-CSRF-TOKEN` and `_csrf`, as in the default implementation offered by Spring Security.

Listing 9.15 The implementation of the `generateToken()` method

```
@Override
public CsrfToken generateToken(HttpServletRequest httpServletRequest) {
  String uuid = UUID.randomUUID().toString();
  return new DefaultCsrfToken("X-CSRF-TOKEN", "_csrf", uuid);
}
```

The `saveToken()` method saves a generated token for a specific client. In the case of the default CSRF protection implementation, the application uses the HTTP session to identify the CSRF token. In our case, we assume that the client has a unique identifier. The client sends the value of its unique ID in the request with the header named `X-IDENTIFIER`. In the method logic, we check whether the value exists in the database. If it does, we update the database with the new value of the token. If not, we create a new record for this ID with the new value of the CSRF token. The following listing presents the implementation of the `saveToken()` method.

Listing 9.16 The implementation of the `saveToken()` method

```
@Override
public void saveToken(
  CsrfToken csrfToken,
  HttpServletRequest httpServletRequest,
  HttpServletResponse httpServletResponse) {
  String identifier =
    httpServletRequest.getHeader("X-IDENTIFIER");
```

```
    Optional<Token> existingToken =                    ①
jpaTokenRepository.findTokenByIdentifier(identifier);

    if (existingToken.isPresent()) {                   ②
      Token token = existingToken.get();
      token.setToken(csrfToken.getToken());
    } else {                                           ③
      Token token = new Token();
      token.setToken(csrfToken.getToken());
      token.setIdentifier(identifier);
      jpaTokenRepository.save(token);
    }
}
```

① Obtains the token from the database by client ID

② If the ID exists, updates the value of the token with a newly generated value

③ If the ID doesn't exist, creates a new record for the ID with a generated value for the CSRF token

The `loadToken()` method implementation loads the token details (if these exist) or returns null otherwise. The following listing shows this implementation.

Listing 9.17 The implementation of the `loadToken()` method

```
@Override
public CsrfToken loadToken(
  HttpServletRequest httpServletRequest) {

  String identifier = httpServletRequest.getHeader("X-IDENTIFIER");

  Optional<Token> existingToken =
    jpaTokenRepository
      .findTokenByIdentifier(identifier);

  if (existingToken.isPresent()) {
    Token token = existingToken.get();
    return new DefaultCsrfToken(
              "X-CSRF-TOKEN",
              "_csrf",
              token.getToken());
  }

  return null;
}
```

We use a custom implementation of the `CsrfTokenRepository` to declare a bean in the configuration class. We then plug the bean into the CSRF protection mechanism with the `csrfTokenRepository()` method of `CsrfConfigurer`. The next listing defines this configuration class.

Listing 9.18 The configuration class for the custom `CsrfTokenRepository`

```
@Configuration
public class ProjectConfig {
```

```
  private final CustomCsrfTokenRepository customTokenRepository;

  // Omitted constructor

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.csrf(c -> {
      c.csrfTokenRepository(customTokenRepository);     ①
    });

    http.authorizeHttpRequests(
      c -> c.anyRequest().permitAll()
    );

    return http.build();
  }

}
```

① Uses the Customizer<CsrfConfigurer<HttpSecurity>> object to plug the new CsrfTokenRepository implementation into the CSRF protection mechanism

The last piece we need to plug in for everything to work well is a `CsrfTokenRequestHandler`. Fortunately, we can use an implementation that Spring Security provides—the `CsrfTokenRequestAttributeHandler`. This implementation simply uses the `generateToken()` method of the `CsrfTokenRepository` to generate a new token when an endpoint is called using the HTTP GET method. It then adds the generated `CsrfToken` upon the request as an attribute.

You can customize the simple behavior of the `CsrfTokenRequestAttributeHandler` object by extending its class. For example, the default implementation Spring Security uses (named `XorCsrfTokenRequestAttributeHandler`) has a more complex behavior. This implementation generates a random value using a `SecureRandom` object and then mixes its byte array with the token generated by the `CsrfTokenRepository` using an XOR logic operation.

However, to avoid adding too much complexity to our example and allow you focus on the configuration part, we will set up a simple `CsrfTokenRequestAttributeHandler` to handle the management of the CSRF token on the HTTP request object. The next listing shows you how to configure the `CsrfTokenRequestAttributeHandler` in the configuration class.

Listing 9.19 The configuration class for the custom `CsrfTokenRepository`

```
@Configuration
public class ProjectConfig {

  private final CustomCsrfTokenRepository customTokenRepository;

  // Omitted constructor

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.csrf(c -> {
      c.csrfTokenRepository(customTokenRepository);
      c.csrfTokenRequestHandler(
        new CsrfTokenRequestAttributeHandler()    ①
```

```
    );                                  ①
   });                                  ①

   http.authorizeHttpRequests(
    c -> c.anyRequest().permitAll()
   );

   return http.build();
  }

}
```

① Setting the CsrfTokenRequestAttributeHandler object to manage the setup of the CSRF token on the HTTP request

In the definition of the controller class presented in listing 9.9, we also add an endpoint that uses the HTTP GET method. We need this method to obtain the CSRF token when testing our implementation:

```
@GetMapping("/hello")
public String getHello() {
 return "Get Hello!";
}
```

You can now start the application and test the new implementation for managing the token. We call the endpoint using HTTP GET to obtain a value for the CSRF token. When making the call, we must use the client's ID within the `X-IDENTIFIER` header, as assumed from the requirement. A new value of the CSRF token is generated and stored in the database. Here's the call:

```
curl -H "X-IDENTIFIER:12345" http://localhost:8080/hello
Get Hello!
```

If you search the token table in the database, you find that the application added a new record for the client with identifier 12345. In my case, the generated value for the CSRF token, which I can see in the database, is `2bc652f5-258b-4a26-b456-928e9bad71f8`. We use this value to call the /hello endpoint with the HTTP POST method, like the next code snippet presents. Of course, we also must provide the client ID that's used by the application to retrieve the token from the database to compare it with the one we provide in the request:

```
curl -XPOST -H "X-IDENTIFIER:12345" -H "X-CSRF-TOKEN:2bc652f5-258b-4a26-b456-928e9bad71f8" http://localhost:8080/hello
Post Hello!
```

Figure 9.11 describes the flow.

If we try to call the /hello endpoint with POST without providing the needed headers, we get a response back with the HTTP status 403 Forbidden. To confirm this, call the endpoint with

```
curl -XPOST http://localhost:8080/hello
```

The response body is

```
{
 "status":403,
 "error":"Forbidden",
```

```
  "message":"Forbidden",
  "path":"/hello"
}
```

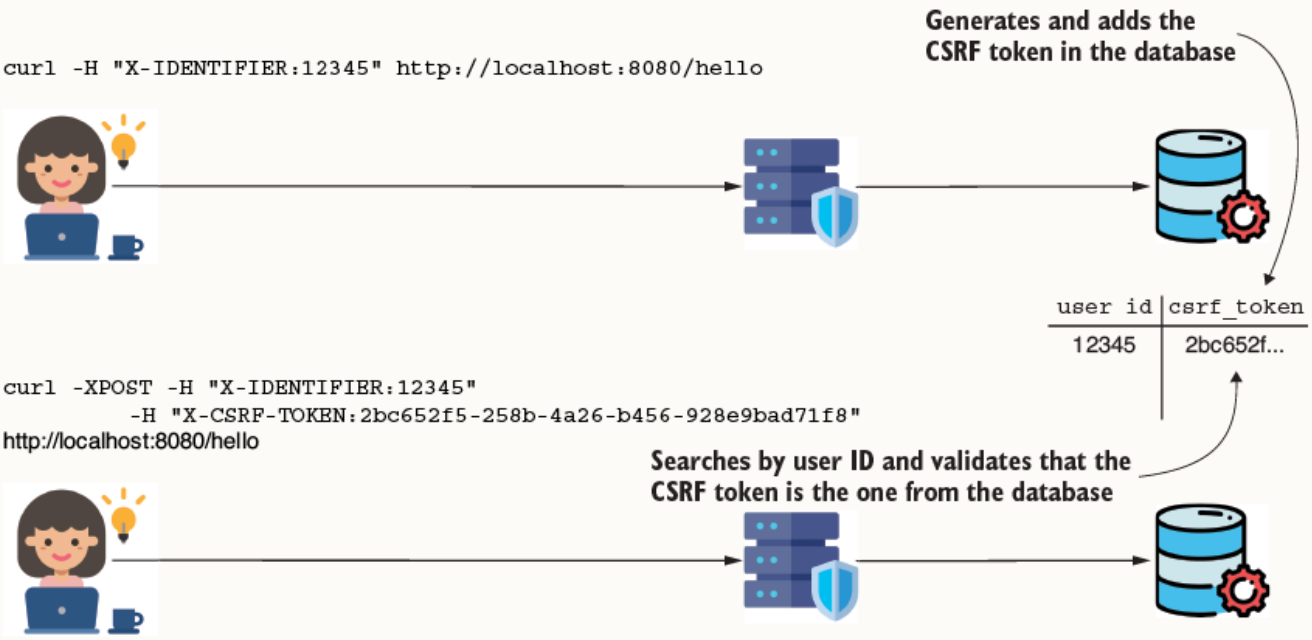


Figure 9.11 First, the GET request generates the CSRF token and stores its value in the database. Any following POST request must send this value. Then the `CsrfFilter` checks whether the value in the request corresponds with the one in the database. Based on this, the request is accepted or rejected.

Summary

- A CSRF is a type of attack where the user is tricked into accessing a page containing a forgery script. This script can impersonate a user logged into an application and execute actions on their behalf.
- CSRF protection is by default enabled in Spring Security.
- The entry point of CSRF protection logic in the Spring Security architecture is an HTTP filter.
- You can customize the capability that offers CSRF protection. Spring Security offers three simple contracts that you can implement and plug in to define custom CSRF protection capabilities:
  - `CsrfToken` —Describes the CSRF token itself
  - `CsrfTokenRepository` —Describes the object that creates, stores, and loads CSRF tokens
  - `CsrfTokenRequestHandler` —Describes the object that manages the way in which the generated CSRF token is set upon the HTTP request

Previous chapter
8 Configuring endpoint-level authorization: Applying restrictions
Next chapter
10 Configuring CORS